

Contour Detection, Polygonal Approximation, and Hough Transform

Teaching Notes for Computer Vision

Contents

1	Introduction	4
2	From Edge Detection to Contours	4
3	Pixel Connectivity	5
3.1	4-Connectivity	5
3.2	8-Connectivity	5
4	Boundary Pixels	6
5	Contour Tracing	6
6	Outer Contours and Holes	6
7	Geometric Properties of a Contour	7
7.1	Contour Length	7
7.2	Contour Area	8
8	Polygonal Approximation	8
9	The Mathematical Objective of Polygonal Approximation	9
10	Ramer–Douglas–Peucker Algorithm	9
11	Step 1: Construct the Approximation Line	9
12	Step 2: Calculate Point-to-Line Distance	10
13	Step 3: Find the Maximum Deviation	10
14	Step 4: Compare with the Tolerance	11
15	Recursive Mathematical Formulation	11
16	Effect of the Approximation Parameter	12

17 Example: Rectangle	12
18 Contour-Based Shape Measurements	13
19 Motivation for the Hough Transform	14
20 Hough Transform for Lines	14
21 A Point in Image Space	15
22 Two Points and a Common Line	15
23 Accumulator Voting	16
24 Problem with Slope–Intercept Form	16
25 Normal Representation of a Line	17
26 Derivation of the Normal Form	17
27 A Point in Normal Hough Space	18
28 Why Lines Produce Peaks	18
29 Discrete Hough Transform	19
30 Hough Transform for Circles	19
31 A Point Voting for a Circle	20
32 Circle Detection with Unknown Radius	20
33 Using Gradient Direction for Circle Detection	21
34 Circle Hough Voting with Gradient Information	21
35 General Hough Transform Principle	22
36 Image Space and Parameter Space	22
37 Why Hough Transform Handles Broken Edges	23
38 Limitations of the Hough Transform	23
39 Contour Detection vs. Hough Transform	24

40 Polygon Approximation vs. Hough Transform	24
41 Complete Shape Analysis Pipeline	25
42 Summary	25

1 Introduction

Edge detection provides a set of pixels corresponding to locations where the image intensity changes rapidly. However, an edge map by itself is only a collection of pixels.

For many computer vision applications, we need a higher-level representation of the shape represented by these pixels.

For example, we may want to determine:

- the boundary of an object,
- the area of an object,
- the perimeter of an object,
- the number of corners or vertices,
- whether an object is approximately circular,
- whether an object is approximately rectangular,
- whether a set of pixels forms a straight line or circle.

Three important classical techniques are useful for this purpose:

1. **Contour detection** — extracts connected object boundaries.
2. **Polygonal approximation** — simplifies a contour using a smaller number of representative points.
3. **Hough transform** — detects predefined geometric shapes by voting in parameter space.

The conceptual progression is

$$\boxed{\text{Image} \rightarrow \text{Edges} \rightarrow \text{Contours} \rightarrow \text{Geometric Representation}} \quad (1)$$

For parametric geometric primitives, an alternative path is

$$\boxed{\text{Image} \rightarrow \text{Edges} \rightarrow \text{Hough Voting} \rightarrow \text{Geometric Primitive}} \quad (2)$$

2 From Edge Detection to Contours

Suppose that an edge detector such as Canny produces a binary edge image

$$E(x, y) \in \{0, 1\}. \quad (3)$$

Here,

$$E(x, y) = 1 \quad (4)$$

indicates that the pixel is classified as an edge pixel.

Although the edge map contains useful information, it does not necessarily tell us which pixels belong to the same boundary.

For example, consider a rectangular object. The edge map may contain four collections of pixels corresponding to the four sides of the rectangle.

We would like to transform these pixels into an ordered sequence:

$$\boxed{p_0 \rightarrow p_1 \rightarrow p_2 \rightarrow \cdots \rightarrow p_{N-1}} \quad (5)$$

This ordered representation is called a **contour**.

A contour can therefore be represented as

$$\boxed{\mathcal{C} = \{p_0, p_1, \dots, p_{N-1}\}} \quad (6)$$

where

$$p_i = (x_i, y_i). \quad (7)$$

The ordering is important because consecutive points correspond to neighboring locations along the boundary.

3 Pixel Connectivity

Contour detection relies heavily on the concept of pixel connectivity.

3.1 4-Connectivity

For a pixel (x, y) , the 4-connected neighborhood is

$$\boxed{\mathcal{N}_4(x, y) = \{(x-1, y), (x+1, y), (x, y-1), (x, y+1)\}} \quad (8)$$

Only horizontal and vertical neighbors are considered.

3.2 8-Connectivity

The 8-connected neighborhood additionally includes diagonal pixels:

$$\boxed{\mathcal{N}_8(x, y) = \{(x+i, y+j) : i, j \in \{-1, 0, 1\}, (i, j) \neq (0, 0)\}} \quad (9)$$

Therefore, two pixels can be connected horizontally, vertically, or diagonally.

In practical image processing, 8-connectivity is often useful for boundary tracing because diagonal boundary pixels should usually be considered part of the same contour.

4 Boundary Pixels

Let Ω denote the set of foreground pixels:

$$\Omega = \{(x, y) : B(x, y) = 1\}. \quad (10)$$

A foreground pixel is considered a boundary pixel if at least one of its neighboring pixels belongs to the background.

Using a neighborhood $\mathcal{N}$, we can write

$$\partial\Omega = \{p \in \Omega : \mathcal{N}(p) \cap \Omega^c \neq \emptyset\}. \quad (11)$$

Here,

$$\Omega^c \quad (12)$$

denotes the background.

Thus, the boundary is the set of foreground pixels that touch the background.

5 Contour Tracing

Finding boundary pixels is not sufficient. We also need to order them.

A contour tracing algorithm starts from a boundary pixel and searches for neighboring boundary pixels.

The resulting ordered sequence is

$$\mathcal{C} = \{(x_0, y_0), (x_1, y_1), \dots, (x_{N-1}, y_{N-1})\}. \quad (13)$$

For a closed contour,

$$p_N = p_0. \quad (14)$$

Thus, a contour can be thought of as a closed chain of connected boundary pixels.

The essential idea is

$$\text{Connected boundary pixels} \longrightarrow \text{Ordered contour}. \quad (15)$$

6 Outer Contours and Holes

An object may contain holes.

Consider a ring-shaped object. It has:

- an outer boundary,

- an inner boundary.

Consequently, contours can have a hierarchical relationship.

Conceptually,

$$\boxed{\text{Outer contour} \rightarrow \text{Inner contour}} \quad (16)$$

and more generally,

$$C_0 \rightarrow \{C_1, C_2, \dots, C_k\}. \quad (17)$$

Here C_0 may represent an outer contour and the C_i may represent boundaries enclosed by it.

This is called the **contour hierarchy**.

Contour hierarchy becomes important when distinguishing between:

- an object,
- a hole inside the object,
- an object inside another object.

7 Geometric Properties of a Contour

Once a contour has been extracted, we can calculate several useful geometric properties.

7.1 Contour Length

Suppose

$$\mathcal{C} = \{p_0, p_1, \dots, p_{N-1}\} \quad (18)$$

where

$$p_i = (x_i, y_i). \quad (19)$$

The Euclidean distance between two consecutive points is

$$\|p_{i+1} - p_i\| = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}. \quad (20)$$

Therefore, the contour perimeter is approximately

$$\boxed{P = \sum_{i=0}^{N-2} \|p_{i+1} - p_i\| + \|p_0 - p_{N-1}\|}. \quad (21)$$

7.2 Contour Area

For a closed contour, the area can be calculated using the **shoelace formula**.

Let

$$p_i = (x_i, y_i). \quad (22)$$

Then

$$A = \frac{1}{2} \left| \sum_{i=0}^{N-1} (x_i y_{i+1} - x_{i+1} y_i) \right| \quad (23)$$

where

$$x_N = x_0, \quad y_N = y_0. \quad (24)$$

This allows the area to be calculated directly from the ordered contour points.

8 Polygonal Approximation

A contour extracted from a digital image may contain hundreds or thousands of points.

For example,

$$\mathcal{C} = \{p_0, p_1, \dots, p_{999}\}. \quad (25)$$

However, the actual object may have a much simpler geometric description.

A rectangle may contain hundreds of contour pixels, but its shape can be described using only four vertices.

Therefore, we seek a simplified representation

$$\boxed{\mathcal{C} \longrightarrow \mathcal{P}} \quad (26)$$

where

$$\mathcal{P} = \{q_0, q_1, \dots, q_{M-1}\} \quad (27)$$

and

$$M \ll N. \quad (28)$$

The goal is to reduce the number of points while preserving the overall shape.

This process is called **polygonal approximation**.

9 The Mathematical Objective of Polygonal Approximation

Polygonal approximation can be viewed as an optimization problem.

We want to minimize the complexity of the representation while keeping the approximation error small.

Conceptually,

$$\boxed{\text{Minimize number of vertices} \quad \text{subject to} \quad \text{approximation error} \leq \epsilon.} \quad (29)$$

Here ϵ is the maximum allowed geometric error.

A commonly used solution is the **Ramer–Douglas–Peucker (RDP) algorithm**.

10 Ramer–Douglas–Peucker Algorithm

The RDP algorithm follows a simple recursive idea.

Given a contour segment:

$$P = \{p_0, p_1, \dots, p_n\}, \quad (30)$$

we initially connect the first and last points with a straight line.

Then:

1. Calculate the perpendicular distance of every intermediate contour point from the line.
2. Find the point having maximum distance.
3. Compare this maximum distance with a tolerance ϵ .
4. If the maximum distance is below ϵ , discard all intermediate points.
5. Otherwise, retain the point with maximum distance and recursively process the two resulting segments.

11 Step 1: Construct the Approximation Line

Consider the endpoints

$$p_0 = (x_0, y_0) \quad (31)$$

and

$$p_n = (x_n, y_n). \quad (32)$$

The first approximation is the line joining these two points.

The line can be written in the general form

$$Ax + By + C = 0. \quad (33)$$

For the line passing through p_0 and p_n , choose

$$A = y_0 - y_n, \quad (34)$$

$$B = x_n - x_0, \quad (35)$$

and

$$C = x_0y_n - x_ny_0. \quad (36)$$

Therefore,

$$\boxed{(y_0 - y_n)x + (x_n - x_0)y + (x_0y_n - x_ny_0) = 0.} \quad (37)$$

12 Step 2: Calculate Point-to-Line Distance

Consider an intermediate contour point

$$p_i = (x_i, y_i). \quad (38)$$

The perpendicular distance from this point to the line

$$Ax + By + C = 0 \quad (39)$$

is

$$\boxed{d_i = \frac{|Ax_i + By_i + C|}{\sqrt{A^2 + B^2}}.} \quad (40)$$

Substituting the values of A , B , and C gives

$$\boxed{d_i = \frac{|(y_0 - y_n)x_i + (x_n - x_0)y_i + x_0y_n - x_ny_0|}{\sqrt{(y_0 - y_n)^2 + (x_n - x_0)^2}}.} \quad (41)$$

This is the fundamental distance calculation used by RDP.

13 Step 3: Find the Maximum Deviation

Calculate

$$d_1, d_2, \dots, d_{n-1}. \quad (42)$$

Then find

$$\boxed{d_{\max} = \max_i d_i.} \quad (43)$$

Let p_k be the point corresponding to this maximum:

$$d_k = d_{\max}. \quad (44)$$

This is the point where the contour deviates most strongly from the current straight-line approximation.

14 Step 4: Compare with the Tolerance

Let ϵ denote the allowed approximation error.

If

$$\boxed{d_{\max} \leq \epsilon,} \quad (45)$$

then the straight line is sufficiently accurate.

Therefore,

$$\boxed{\{p_0, p_1, \dots, p_n\} \longrightarrow \{p_0, p_n\}.} \quad (46)$$

All intermediate points can be discarded.

On the other hand, if

$$\boxed{d_{\max} > \epsilon,} \quad (47)$$

then p_k must be retained.

The contour is split into

$$P_1 = \{p_0, \dots, p_k\} \quad (48)$$

and

$$P_2 = \{p_k, \dots, p_n\}. \quad (49)$$

The same procedure is then applied recursively to both segments.

15 Recursive Mathematical Formulation

The RDP procedure can be expressed mathematically as

$$\text{RDP}(P, \epsilon) = \begin{cases} \{p_0, p_n\}, & d_{\max} \leq \epsilon, \\ \text{RDP}(P_1, \epsilon) \cup \text{RDP}(P_2, \epsilon), & d_{\max} > \epsilon. \end{cases} \quad (50)$$

The recursion continues until every segment satisfies

$$d_{\max} \leq \epsilon. \quad (51)$$

The final result is

$$\boxed{\mathcal{C} \longrightarrow \mathcal{P} = \{q_0, q_1, \dots, q_{M-1}\}} \quad (52)$$

with

$$M \ll N. \quad (53)$$

16 Effect of the Approximation Parameter

The parameter ϵ controls the degree of simplification.

For a small tolerance,

$$\epsilon \downarrow \quad (54)$$

we obtain

$$\boxed{\text{More vertices} \quad \Rightarrow \quad \text{higher fidelity.}} \quad (55)$$

For a large tolerance,

$$\epsilon \uparrow \quad (56)$$

we obtain

$$\boxed{\text{Fewer vertices} \quad \Rightarrow \quad \text{greater simplification.}} \quad (57)$$

Therefore,

$$\boxed{\text{Geometric accuracy} \quad \longleftrightarrow \quad \text{Representation complexity.}} \quad (58)$$

17 Example: Rectangle

Suppose the contour of a rectangle contains 400 pixels.

The original representation is

$$\mathcal{C} = \{p_0, p_1, \dots, p_{399}\}. \quad (59)$$

After polygonal approximation, the result may be

$$\mathcal{P} = \{q_0, q_1, q_2, q_3\}. \quad (60)$$

Thus,

$$400 \longrightarrow 4. \quad (61)$$

The simplified representation retains the essential geometry of the rectangle while dramatically reducing the number of points.

18 Contour-Based Shape Measurements

Once a contour is available, several useful measurements can be computed.

These include:

- area,
- perimeter,
- bounding box,
- centroid,
- convexity,
- number of polygon vertices,
- circularity,
- aspect ratio.

One useful measure is circularity:

$$C = \frac{4\pi A}{P^2} \quad (62)$$

where A is the contour area and P is its perimeter.

For a perfect circle,

$$C = 1. \quad (63)$$

For more irregular shapes,

$$C < 1. \quad (64)$$

19 Motivation for the Hough Transform

Contour-based techniques depend on connected boundary pixels.

However, real images frequently contain:

- noise,
- broken edges,
- occlusion,
- missing pixels,
- disconnected edge fragments.

Suppose we want to detect a straight line.

The line may appear as several disconnected edge fragments:

$$\boxed{\text{edge fragment} \quad \text{gap} \quad \text{edge fragment.}} \quad (65)$$

Contour tracing may fail to interpret these fragments as one object.

The Hough transform uses a different idea:

Find the geometric model that receives the largest number of votes from the observed points.

Thus,

$$\boxed{\text{Image space} \rightarrow \text{Parameter space} \rightarrow \text{Voting} \rightarrow \text{Peaks.}} \quad (66)$$

20 Hough Transform for Lines

The simplest equation for a straight line is

$$y = mx + c. \quad (67)$$

Here,

$$m = \text{slope} \quad (68)$$

and

$$c = \text{intercept.} \quad (69)$$

Therefore, a line is described by two parameters:

$$\boxed{(m, c).} \quad (70)$$

21 A Point in Image Space

Consider an edge point

$$(x_i, y_i). \quad (71)$$

Every line passing through this point satisfies

$$y_i = mx_i + c. \quad (72)$$

Rearranging,

$$\boxed{c = y_i - mx_i.} \quad (73)$$

Now x_i and y_i are fixed.

Therefore, as m varies, c changes according to

$$c = y_i - mx_i. \quad (74)$$

This is a straight line in (m, c) parameter space.

Thus,

$$\boxed{\text{One point in image space} \longrightarrow \text{one line in parameter space.}} \quad (75)$$

22 Two Points and a Common Line

Consider two points

$$p_1 = (x_1, y_1) \quad (76)$$

and

$$p_2 = (x_2, y_2). \quad (77)$$

Each point generates a line in Hough parameter space:

$$c = y_1 - mx_1 \quad (78)$$

and

$$c = y_2 - mx_2. \quad (79)$$

If the two image points lie on the same physical line, the two parameter-space lines intersect at the parameters corresponding to that physical line.

Therefore,

$$\boxed{\text{Collinear points} \longrightarrow \text{intersection in parameter space.}} \quad (80)$$

For many points belonging to the same line, many parameter-space curves intersect at approximately the same point.

This creates a peak.

23 Accumulator Voting

The Hough transform implements this intersection idea using an accumulator.

Let

$$A(m, c) \quad (81)$$

be the accumulator.

Initially,

$$A(m, c) = 0. \quad (82)$$

For each edge pixel (x_i, y_i) , all possible line parameters satisfying

$$c = y_i - mx_i \quad (83)$$

receive votes:

$$\boxed{A(m, c) \leftarrow A(m, c) + 1.} \quad (84)$$

After all edge pixels have voted, a line corresponds to a strong peak:

$$\boxed{(m^*, c^*) = \arg \max_{m, c} A(m, c).} \quad (85)$$

24 Problem with Slope–Intercept Form

The representation

$$y = mx + c \quad (86)$$

has a numerical problem.

For a vertical line,

$$x = c \quad (87)$$

and the slope is effectively infinite:

$$m \rightarrow \infty. \quad (88)$$

Therefore, a better representation is needed.

The Hough transform commonly uses the normal representation.

25 Normal Representation of a Line

A line can be represented as

$$\boxed{x \cos \theta + y \sin \theta = \rho.} \quad (89)$$

Here,

$$\rho = \text{perpendicular distance from the origin to the line} \quad (90)$$

and

$$\theta = \text{orientation of the normal vector.} \quad (91)$$

The normal vector is

$$\boxed{\mathbf{n} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}.} \quad (92)$$

26 Derivation of the Normal Form

Consider a point

$$\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix} \quad (93)$$

on the line.

The projection of $\mathbf{p}$ onto the unit normal vector is

$$\mathbf{n}^T \mathbf{p}. \quad (94)$$

Since this projection is the perpendicular distance ρ ,

$$\mathbf{n}^T \mathbf{p} = \rho. \quad (95)$$

Substituting

$$\mathbf{n} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \quad (96)$$

gives

$$\begin{bmatrix} \cos \theta & \sin \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \rho. \quad (97)$$

Therefore,

$$\boxed{x \cos \theta + y \sin \theta = \rho.} \quad (98)$$

This representation works for vertical as well as horizontal lines.

27 A Point in Normal Hough Space

Consider an edge point

$$(x_i, y_i). \quad (99)$$

Any line passing through this point must satisfy

$$x_i \cos \theta + y_i \sin \theta = \rho. \quad (100)$$

Therefore,

$$\boxed{\rho(\theta) = x_i \cos \theta + y_i \sin \theta.} \quad (101)$$

As θ varies, ρ traces a sinusoidal curve.

Thus,

$$\boxed{\text{Point in image space} \longrightarrow \text{Sinusoid in Hough space.}} \quad (102)$$

28 Why Lines Produce Peaks

Suppose several image points belong to the same line.

For each point,

$$\rho_i = x_i \cos \theta + y_i \sin \theta. \quad (103)$$

At the correct line parameters

$$(\theta^*, \rho^*), \quad (104)$$

all these sinusoidal curves intersect.

Thus,

$$\boxed{\text{Many collinear points} \longrightarrow \text{many votes at one parameter location.}} \quad (105)$$

Therefore, a line is detected by a peak in the accumulator:

$$\boxed{(\theta^*, \rho^*) = \arg \max_{\theta, \rho} A(\theta, \rho).} \quad (106)$$

29 Discrete Hough Transform

In practice, the parameter space is discretized.

Let

$$\theta \in \{\theta_1, \theta_2, \dots, \theta_K\} \quad (107)$$

and

$$\rho \in \{\rho_1, \rho_2, \dots, \rho_L\}. \quad (108)$$

The accumulator becomes a matrix

$$A[k, l]. \quad (109)$$

For every edge point (x_i, y_i) and every angle θ_k ,

$$\rho = x_i \cos \theta_k + y_i \sin \theta_k. \quad (110)$$

The corresponding accumulator bin receives one vote:

$$\boxed{A[k, l] \leftarrow A[k, l] + 1.} \quad (111)$$

After all points have voted, local maxima of the accumulator represent candidate lines.

30 Hough Transform for Circles

The equation of a circle is

$$\boxed{(x - a)^2 + (y - b)^2 = r^2.} \quad (112)$$

The parameters are:

$$a = \text{x-coordinate of center}, \quad (113)$$

$$b = \text{y-coordinate of center}, \quad (114)$$

and

$$r = \text{radius}. \quad (115)$$

Therefore, a circle requires three parameters:

$$(a, b, r). \quad (116)$$

Consequently, the Hough accumulator becomes

$$A(a, b, r). \quad (117)$$

31 A Point Voting for a Circle

Consider an edge point

$$(x_i, y_i). \quad (118)$$

For this point to lie on a circle,

$$(x_i - a)^2 + (y_i - b)^2 = r^2. \quad (119)$$

If r is fixed, this equation describes a circle in the (a, b) parameter space.

Therefore,

$$\boxed{\text{Image point} \longrightarrow \text{Circle in center parameter space.}} \quad (120)$$

If several image points belong to the same circle, their parameter-space circles intersect at the true center.

32 Circle Detection with Unknown Radius

If the radius is unknown, the parameter space becomes

$$(a, b, r). \quad (121)$$

Each image point satisfies

$$(x_i - a)^2 + (y_i - b)^2 = r^2. \quad (122)$$

Therefore, every edge point votes for many possible combinations of center and radius.

The accumulator is

$$A(a, b, r). \quad (123)$$

The detected circle corresponds to the maximum:

$$\boxed{(a^*, b^*, r^*) = \arg \max_{a, b, r} A(a, b, r).} \quad (124)$$

33 Using Gradient Direction for Circle Detection

The basic circle Hough transform can be computationally expensive because every edge point can vote for many possible centers.

The gradient provides additional information.

For a circle, the image gradient at the boundary is approximately radial.

Suppose an edge point is

$$p = (x, y) \tag{125}$$

and its gradient direction is

$$\theta. \tag{126}$$

The center of the circle should lie along the gradient direction.

Therefore, for a known radius r ,

$$a = x - r \cos \theta \tag{127}$$

and

$$b = y - r \sin \theta. \tag{128}$$

Depending on whether the gradient points inward or outward, the alternative direction is

$$a = x + r \cos \theta, \quad b = y + r \sin \theta. \tag{129}$$

Thus, gradient information reduces the set of possible circle centers.

34 Circle Hough Voting with Gradient Information

For an edge point (x, y) , gradient direction θ , and known radius r , the possible center is approximately

$$(a, b) = (x \pm r \cos \theta, y \pm r \sin \theta). \tag{130}$$

The corresponding accumulator bin receives a vote:

$$A(a, b) \leftarrow A(a, b) + 1. \tag{131}$$

After processing all edge points,

$$(a^*, b^*) = \arg \max_{a, b} A(a, b) \tag{132}$$

gives the estimated center.

If multiple radii are considered, the accumulator becomes

$$A(a, b, r). \quad (133)$$

35 General Hough Transform Principle

The line and circle examples illustrate a general principle.

Suppose a geometric primitive is represented using parameters

$$\theta = (\theta_1, \theta_2, \dots, \theta_d). \quad (134)$$

For every edge point, determine which parameter values could have generated that point.

Each edge point contributes votes:

$$\boxed{A(\theta) \leftarrow A(\theta) + 1.} \quad (135)$$

After all points have voted, geometric primitives correspond to peaks in parameter space:

$$\boxed{\theta^* = \arg \max_{\theta} A(\theta).} \quad (136)$$

Therefore,

$$\boxed{\text{Hough Transform} = \text{Voting in parameter space.}} \quad (137)$$

36 Image Space and Parameter Space

The distinction between image space and parameter space is central to understanding the Hough transform.

	Image Space	Hough Space
Representation	Pixels (x, y)	Shape parameters
Line	Points forming a line	Peak (ρ, θ)
Circle	Boundary points	Peak (a, b, r)
Noise	Scattered points	Diffuse votes
True geometric structure	Aligned boundary points	Concentrated peak

The essential transformation is

$$\boxed{\text{Geometric structure in image space} \longrightarrow \text{Concentration of votes in parameter space.}} \quad (138)$$

37 Why Hough Transform Handles Broken Edges

Suppose a straight line is partially occluded.

The visible edge may consist of two disconnected fragments.

Contour tracing may treat them as two separate contours.

However, every visible point belonging to the line still approximately satisfies

$$x \cos \theta + y \sin \theta = \rho. \quad (139)$$

Consequently, both fragments vote for approximately the same parameter pair (ρ, θ) .

Thus,

Disconnected edge fragments $\longrightarrow$ Common peak in parameter space.

 (140)

This is one of the major advantages of the Hough transform.

38 Limitations of the Hough Transform

The Hough transform also has important limitations.

1. The parameter space can become very large.
2. Computational cost increases with the number of parameters.
3. The resolution of the accumulator affects detection accuracy.
4. Noise can produce spurious peaks.
5. Multiple nearby shapes may produce overlapping peaks.
6. Threshold selection for identifying peaks is important.

For lines,

$$A(\rho, \theta) \quad (141)$$

is two-dimensional.

For circles with unknown radius,

$$A(a, b, r) \quad (142)$$

is three-dimensional.

More complicated geometric models may require even higher-dimensional parameter spaces.

39 Contour Detection vs. Hough Transform

Although both techniques operate on edge information, they solve different problems.

	Contour Detection	Hough Transform
Main idea	Follow connected boundaries	Vote for geometric models
Connectivity	Usually important	Not required
Output	Ordered contour points	Geometric parameters
Good for	Arbitrary shapes	Parametric shapes
Handles gaps	Less robust	More robust
Representation	Boundary-based	Model-based
Examples	Objects, polygons	Lines, circles, ellipses

Thus,

$$\boxed{\text{Contour Detection} \rightarrow \text{“Where is the boundary?”}} \quad (143)$$

whereas

$$\boxed{\text{Hough Transform} \rightarrow \text{“Which geometric model explains the boundary?”}} \quad (144)$$

40 Polygon Approximation vs. Hough Transform

These two approaches represent different forms of geometric abstraction.

Polygonal approximation asks:

$$\boxed{\text{Can the boundary be represented using fewer vertices?}} \quad (145)$$

Hough transform asks:

$$\boxed{\text{Can the boundary be explained by a known parametric model?}} \quad (146)$$

For example, a circular contour can be represented using polygonal approximation as

$$\text{Circle} \rightarrow \text{Many contour points} \rightarrow \text{Polygon}. \quad (147)$$

Alternatively,

$$\text{Circle} \rightarrow \text{Edge points} \rightarrow \text{Hough transform} \rightarrow (a, b, r). \quad (148)$$

The first approach is a form of

$$\boxed{\text{non-parametric geometric simplification,}} \quad (149)$$

while the second is

$$\boxed{\text{parametric model detection.}} \quad (150)$$

41 Complete Shape Analysis Pipeline

The concepts developed in this chapter can be combined into a classical computer vision pipeline.

$$\begin{array}{l}
 \text{Image} \rightarrow \text{Edge Detection} \\
 \rightarrow \text{Binary Edge Map} \\
 \rightarrow \text{Contour Detection} \\
 \rightarrow \text{Contour Representation} \\
 \rightarrow \left\{ \begin{array}{l} \text{Polygonal Approximation} \\ \text{Geometric Measurements} \\ \text{Shape Classification} \end{array} \right.
 \end{array} \tag{151}$$

For parametric geometric primitives, an alternative pipeline is

$$\begin{array}{l}
 \text{Image} \rightarrow \text{Edge Detection} \\
 \rightarrow \text{Edge Points} \\
 \rightarrow \text{Hough Voting} \\
 \rightarrow \text{Accumulator Peaks} \\
 \rightarrow \text{Geometric Primitives.}
 \end{array} \tag{152}$$

42 Summary

The major concepts developed in this chapter can be summarized as follows.

Contour Representation

An ordered contour is

$$\mathcal{C} = \{(x_0, y_0), \dots, (x_{N-1}, y_{N-1})\}. \tag{153}$$

Contour Area

$$A = \frac{1}{2} \left| \sum_i (x_i y_{i+1} - x_{i+1} y_i) \right|. \tag{154}$$

Contour Perimeter

$$P = \sum_i \|p_{i+1} - p_i\|. \tag{155}$$

Polygonal Approximation

The point-to-line distance is

$$d = \frac{|Ax + By + C|}{\sqrt{A^2 + B^2}}. \quad (156)$$

The RDP algorithm retains a point when

$$d_{\max} > \epsilon. \quad (157)$$

Line Hough Transform

A line is represented using

$$x \cos \theta + y \sin \theta = \rho. \quad (158)$$

The accumulator is

$$A(\rho, \theta). \quad (159)$$

The detected line corresponds to

$$(\rho^*, \theta^*) = \arg \max_{\rho, \theta} A(\rho, \theta). \quad (160)$$

Circle Hough Transform

A circle is represented by

$$(x - a)^2 + (y - b)^2 = r^2. \quad (161)$$

The accumulator is

$$A(a, b, r). \quad (162)$$

The detected circle is

$$(a^*, b^*, r^*) = \arg \max_{a, b, r} A(a, b, r). \quad (163)$$

The overall conceptual progression is

$$\text{Pixels} \rightarrow \text{Edges} \rightarrow \text{Contours} \rightarrow \text{Geometric Representation}. \quad (164)$$

For known geometric primitives,

$$\text{Edge Points} \rightarrow \text{Parameter Space} \rightarrow \text{Voting} \rightarrow \text{Geometric Model}. \quad (165)$$